

Notes on Hyperparameters, Bias-Variance Tradeoff, and Cross-Validation

Luis Mendez

HYPERPARAMETERS, RECAP

Parameters are learned by the algorithm from the training data, e.g., the weights of a linear model or the splits of a decision tree. Hyperparameters are set before training starts and are not learned directly from the data, instead they control how the model learns and how flexible (how much capacity) it is allowed to be. Examples include the regularization strength λ in linear/logistic regression, the learning rate α in gradient-based methods, *max_depth* and *min_samples_leaf* in trees, *n_estimators* in random forests, and the kernel/ C / γ in SVMs.

Because hyperparameters shape model capacity, they are the main lever we use to move along the bias-variance tradeoff described below. Picking them well is the entire point of hyperparameter optimization (grid search, random search, Bayesian optimization, etc.), but any of these search strategies is only as good as the way we *evaluate* a given hyperparameter combination, which is where cross-validation comes in.

BIAS-VARIANCE TRADEOFF

For a model $\hat{f}$ trained to approximate the true function f , the expected squared error on an unseen point x can be decomposed as

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}(\hat{f}(x))^2 + \text{Var}(\hat{f}(x)) + \sigma^2, \quad (1)$$

where σ^2 is the irreducible noise in the data (error we can never remove, regardless of the model).

- **Bias** is the error introduced by approximating a real, possibly complicated, relationship with a simplified model. High bias means the model is not flexible enough to capture the underlying pattern, it makes strong assumptions about the data (e.g., a linear model trying to fit a non-linear relationship). High bias shows up as **underfitting**: poor performance on both the training set and the test set.
- **Variance** is how much the model's predictions would change if trained on a different sample of the training data. High variance means the model is too sensitive to the specific training set, it fits noise as if it were signal. High variance shows up as **overfitting**: very good performance on the training set but poor performance on the test set.

Increasing model capacity (deeper trees, more estimators, smaller regularization λ , more polynomial degrees) tends to decrease bias but increase variance, and vice versa. There is no free lunch, we cannot reduce both at the same time indefinitely,

we are looking for the sweet spot that minimizes the total error ($\text{bias}^2 + \text{variance}$), not just one of the two terms.

This is precisely why hyperparameters matter: *max_depth*, λ , *n_estimators*, learning rate, etc. are all knobs that move a model along this tradeoff. Hyperparameter optimization is, in essence, an automated search for the point on the bias-variance curve that generalizes best.

Diagnosing bias vs. variance

- Training error high, validation error high, similar to each other $\rightarrow$ high bias (underfitting). Fix: increase model complexity, add features, reduce regularization.
- Training error low, validation error much higher $\rightarrow$ high variance (overfitting). Fix: get more data, reduce model complexity, increase regularization, use techniques like bagging.

CROSS-VALIDATION

To choose hyperparameters (or to estimate how well a model will generalize) we cannot just look at training error, since it will keep improving as we add capacity, masking overfitting. We need an estimate of performance on data the model has not seen. A single train/test split gives one such estimate, but it is noisy: it depends heavily on which observations happened to land in the test set, especially with small datasets. Cross-validation reduces this variance in the performance estimate by repeating the train/validation split multiple times and averaging the results.

K-Fold Cross-Validation

The training data is split into k equally sized folds. For each of the k iterations, one fold is held out as the validation set and the model is trained on the remaining $k - 1$ folds. The performance metric (accuracy, RMSE, F1, etc.) is computed on each held-out fold, and the final estimate is the average across the k folds:

$$\text{CV score} = \frac{1}{k} \sum_{i=1}^k \text{score}_i. \quad (2)$$

Every observation is used for validation exactly once and for training $k - 1$ times. Common choices are $k = 5$ or $k = 10$. Larger k means less bias in the estimate (each training fold is closer in size to the full dataset) but higher variance and more computational cost, since we train the model k times.

Stratified K-Fold

For classification problems, especially with imbalanced classes, plain k-fold can produce folds where the class proportions differ a lot from the overall dataset (or even folds missing a class entirely). Stratified k-fold preserves the percentage of samples of each class in every fold, giving a more reliable estimate.

Leave-One-Out (LOOCV)

A special case of k-fold where $k = n$ (the number of observations). Each fold is a single observation held out for validation. It uses almost all the data for training every time (low bias) but is expensive to compute for large n and can have high variance across the folds since each held-out point is very small a sample.

Repeated / Nested Cross-Validation

- **Repeated k-fold:** run k-fold multiple times with different random splits and average, further reducing the variance of the estimate.
- **Nested cross-validation:** used when we are doing hyperparameter search *and* want an unbiased estimate of generalization error at the same time. An outer loop splits data into train/test for performance estimation, and within each outer training fold an inner loop of cross-validation is used to select the best hyperparameters. This avoids leaking information from the test fold into the hyperparameter choice, which would otherwise give an overly optimistic performance estimate.

Time Series / Grouped Data

Standard k-fold assumes samples are i.i.d. If the data has a time component (e.g., forecasting) or groups that should not be split across folds (e.g., multiple rows per patient), regular k-fold can leak information from the future into training, or leak information between folds. In those cases we should instead use:

- **Time Series Split:** folds respect chronological order, training always happens on the past and validating on the future.
- **Group K-Fold:** ensures the same group never appears in both the training and validation fold.

Cross-Validation and Hyperparameter Search

When we run grid search or random search, each candidate hyperparameter combination is evaluated using cross-validation (i.e., k models are trained and scored per combination, and the average CV score is what is compared across candidates). This is why hyperparameter search can be so expensive: total training runs $\approx$ (number of hyperparameter combinations) $\times k$. It is also why nested cross-validation matters if we report a final performance number, otherwise we risk over-fitting our hyperparameter choice to the validation folds, which is a subtler form of the same overfitting problem the bias-variance tradeoff describes.